

Marian Sorin-Nicu



POLITECNICO
MILANO 1863

Visual Analysis of Bowling: 3D Ball Trajectory and Spin Estimation from a Monocular Video

Deliverable: Project Report
Topic: F11 – Visual Analysis of Sport Events (Bowling)
Author: Marian Sorin-Nicu (ID 11161743)
Supervisor: Prof. Vincenzo Caglioti
Version: 1
Date: July 2026

Contents

Table of Contents	3
List of Figures	4
List of Tables	4
1 Introduction	5
2 Related Work	5
3 Solution Approach	5
3.1 Ball Detection	5
3.2 Lane Calibration and Camera Model	6
3.3 3D Trajectory Reconstruction	7
3.4 Spin Estimation	8
4 Implementation	8
4.1 Software Architecture	8
4.2 Key Implementation Details	9
5 Experimental Results	9
5.1 Video Data	9
5.2 Ball Detection	9
5.3 Lane Calibration	10
5.4 Multi-Throw Generalisation	11
5.5 2D and 3D Trajectory	12
5.6 Spin Estimation	13
6 Conclusions and Open Problems	14
7 References	15

List of Figures

1	Ball detection at release. Green circle = contour method (preferred); yellow = Hough fallback. Ball correctly localised at (938, 680), radius $r = 46$ px. Top 15 % of frame excluded.	10
2	Lane calibration: 6 Hough lines (yellow/orange) restricted to the right 55 % of the frame, converging to the vanishing point (red circle) at (1012, 106). The player is excluded by the ROI mask (vertical blue line).	11
3	Ball trajectory overlaid on a mid-throw frame for 6 different throws from the same broadcast clip ($t = 126$ s to $t = 1126$ s). Colour encodes time: blue (release) → yellow (mid) → red (end). White dots = raw detections. Detection counts range from 59 to 244 per throw.	12
4	2D ball trajectory over 6.36 s. Colour encodes time: blue (release) → yellow (mid) → red (end). White dots = raw detections. Arrows indicate direction of travel. The hook of +168 px is annotated with a yellow brace.	13
5	Lucas–Kanade tracking on the ball surface crop ($5\times$ magnified inset). Coloured arcs trace the trajectory of 17 feature points across 25 frames. The yellow circle marks the detected finger hole.	14

List of Tables

1	Pipeline modules and their responsibilities.	8
2	Ball detection statistics over the throw segment.	9
3	Lane calibration results.	10
4	Pipeline results across 10 throws from the same broadcast clip. Each row is an independent run on a different bowler and camera segment.	11
5	Trajectory analysis results.	12

1 Introduction

Tenpin bowling is a precision sport in which ball dynamics — speed, entry angle, and spin — are the primary determinants of pin-fall outcome. Professional coaching and broadcast analysis both require quantitative characterisation of these properties, yet attaching physical sensors to a competition ball is prohibited by regulations. A camera-based, non-contact measurement system therefore has direct practical value both for athletes and for automated sports analytics.

From a computer vision perspective the problem is non-trivial: the ball is small relative to the frame (radius $\approx 25\text{--}60$ px in a broadcast feed), the surface is often a single dark colour with few trackable features, the lane geometry is partially occluded by the bowler at release, and a single camera provides no stereo baseline.

This project addresses all three measurements – detection, 3D trajectory, and spin – within the constraints of a *single monocular broadcast camera*, using exclusively the algorithms presented in the IACV course. The system takes as input a standard bowling video together with the known lane dimensions (USBC standard: 18.29 m long, 1.05 m wide) and produces:

1. the 2D ball centre per frame in image coordinates,
2. the metric 3D trajectory on the lane plane, including estimated ball height as a function of time,
3. an estimate of ball spin (rotation axis and angular velocity in RPM).

The rest of the report is structured as follows: Section reviews related work; Section presents the solution approach with its mathematical foundation; Section describes the software architecture; Section reports experimental results; Section concludes and discusses open problems.

2 Related Work

Ball tracking in sports video has been studied extensively. *Yan et al.* [5] combine background subtraction and Kalman filtering for cricket ball tracking. *Chu et al.* [6] apply trajectory fitting to table tennis. For bowling specifically, *Gibbs et al.* [7] attach IMU sensors directly to the ball to measure spin, bypassing the vision problem entirely. *Shah et al.* [8] propose the closest camera-based approach, using lane markers as calibration anchors and back-projection for trajectory recovery — which is the general strategy adopted here.

Camera calibration from scene geometry, in particular from vanishing points, is a well-established technique introduced by *Caprile and Torre* [2] and further developed by *Cipolla et al.* [3]. The Lucas–Kanade optical flow tracker [1] remains the standard for sparse feature tracking and is applied here for spin estimation. All of these algorithms are covered in the IACV course slides (Lectures E, F, G, H) and are applied directly without reinvention.

3 Solution Approach

3.1 Ball Detection

Background Subtraction (MOG2)

The ball is the primary moving foreground object on an otherwise static lane. A Mixture-of-Gaussians background model (MOG2, [4]) is warmed up on 3 s of video recorded before the throw. The foreground

mask M_{fg} isolates pixels whose intensity deviates from the learned background distribution beyond a variance threshold $\theta = 50$.

Colour Segmentation in HSV

A parallel HSV colour mask M_c covers ball-typical hues: dark blue/indigo ($H \in [95^\circ, 145^\circ]$), near-black (low saturation), red (two wrap-around ranges), green, and orange. Morphological open/close with a 5×5 elliptic structuring element removes noise and fills gaps. The combined mask is:

$$M = M_{fg} \wedge M_c \quad (1)$$

Contour Detection and Hough Fallback

`cv2.findContours` extracts connected regions from M . For each contour the minimum enclosing circle (c_x, c_y, r) is computed; a candidate is accepted if:

- Radius: $r \in [18, 110]$ px (full depth range on the lane),
- Circularity: $A/(\pi r^2) \geq 0.40$,
- Vertical position: $c_y > 0.15H$ (reject scoreboard overlays in the top 15 % of frame).

The candidate with the highest score $s = \text{circularity} \times A$ is selected. If no contour passes the tests, the **Hough Transform (Circles)** on the bilateral-filtered, colour-masked grayscale image serves as fallback (IACV Lecture H). Temporal consistency is enforced by rejecting detections that jump more than $d_{\max} = 160$ px from the previous frame.

3.2 Lane Calibration and Camera Model

Camera Model

A pinhole camera with zero skew is assumed throughout (IACV Lecture E):

$$\mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}, \quad f_x = f_y = f \quad (2)$$

The principal point (c_x, c_y) is taken at the image centre.

Vanishing Point from Lane Lines

The **Canny Edge Detector** (Gaussian $\rightarrow$ gradient $\rightarrow$ NMS $\rightarrow$ hysteresis) produces a robust edge map from the lane region. The **Hough Transform (Lines)** then votes in the (ρ, θ) accumulator to extract line segments. To avoid contamination from the bowler's body, the search region is restricted to the right 55 % of the frame ($x > 0.45W$), and only lines with inclination $\theta \in [35^\circ, 88^\circ]$ are retained. The top- $k = 6$ longest lines are used. Each pair of lines (l_i, l_j) yields a vanishing point by the homogeneous **Vanishing Point from Lines** formula (IACV Lecture E):

$$\mathbf{v} = \mathbf{l}_i \times \mathbf{l}_j \quad (3)$$

The final lane vanishing point $\mathbf{v}_{\text{lane}}$ is the coordinate-wise median over all pairwise intersections.

Intrinsic Estimation from the IAC

The **IAC/ ω Constraints** algorithm recovers the focal length from the orthogonality condition on the Image of the Absolute Conic (IACV Lecture E). For two orthogonal vanishing points:

$$\mathbf{v}_1^T \boldsymbol{\omega} \mathbf{v}_2 = 0, \quad \boldsymbol{\omega} = \mathbf{K}^{-T} \mathbf{K}^{-1} \quad (4)$$

Under the simplification $f_x = f_y = f$ with the principal point at image centre, this yields the closed-form estimate:

$$f^2 = -(v_{1x} - c_x)(v_{2x} - c_x) - (v_{1y} - c_y)(v_{2y} - c_y) \quad (5)$$

If the discriminant of Eq. (5) is negative (insufficiently reliable vanishing points), the focal length falls back to the ball-diameter scale reference: $f \approx r_{\text{px}} \cdot Z / R_{\text{ball}}$, with $R_{\text{ball}} = 0.108 \text{ m}$.

DLT Homography

Four control points with known metric coordinates on the lane plane — the two foul-line corners and the two nearest arrow markers at 4.57 m — are identified manually in one reference frame. Before applying **DLT for Homography**, both image and lane-plane coordinates are normalised (**Normalisation of Coordinates**: translate to centroid, scale to mean distance $\sqrt{2}$) to reduce numerical instability (IACV Lecture F). The DLT builds the $2n \times 9$ linear system and solves via SVD; the result is denormalised to give $\mathbf{H}$ such that:

$$\mathbf{x} \sim \mathbf{H} \mathbf{X}, \quad \mathbf{X} = (X, Y, 0)^T \quad (6)$$

3.3 3D Trajectory Reconstruction

Triangulation via Rays

Given ball centre $\mathbf{x} = (u, v, 1)^T$ in image coordinates, the back-projected ray is (cheat-sheet: **Triangulation via Rays**):

$$\mathbf{d} = \mathbf{K}^{-1} \mathbf{x} \quad (7)$$

The camera rotation and translation $[\mathbf{R} \mid \mathbf{t}]$ are recovered via **Homography Decomposition** (IACV Lecture F):

$$\mathbf{r}_1 = \mathbf{K}^{-1} \mathbf{h}_1, \quad \mathbf{r}_2 = \mathbf{K}^{-1} \mathbf{h}_2, \quad \mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2 \quad (8)$$

The rotation matrix is enforced to $SO(3)$ via **SVD Orthonormalisation** ($\hat{\mathbf{R}} = \mathbf{U}\mathbf{V}^T$). Intersecting the ray with the lane plane ($Z = 0$ in lane coordinates) yields the metric ground position (X, Y) .

Height Model

The ball travels in a parabolic arc under gravity. A height function $Z(t)$ is estimated by fitting:

$$Z(t) = Z_0 + v_z t - \frac{1}{2} g t^2, \quad g = 9.81 \text{ m/s}^2 \quad (9)$$

using the ball's apparent radius in image space as a depth proxy (known ball diameter = 21.6 cm).

Smoothing

A degree-3 polynomial is fit to the $(X(t), Y(t))$ sequence via least squares, suppressing outlier detections and providing a smooth, differentiable trajectory for speed and hook-angle computation.

3.4 Spin Estimation

Lucas–Kanade Tracking in the Ball-Crop Frame

The ball translates $\approx 100\text{--}150$ px per frame at lane speed, exceeding the LK search window. Translation is removed by cropping the ball region per frame before tracking. Feature points are initialised inside a circular ball mask using the **Harris Corner Detector** (structure tensor + eigenvalue response + NMS). Finger holes are additionally detected as small dark blobs via **Hough Transform (Circles)** with radii $r \in [R/12, R/5]$ and mean crop brightness below $80/255$.

The **Lucas–Kanade** tracker (`cv2.calcOpticalFlowPyrLK`, pyramid levels = 3, window = 21×21) minimises the brightness-constancy error by solving at each pyramid level:

$$\left[\sum_{\mathbf{x}} (\nabla I)(\nabla I)^T \right] \mathbf{d} = - \sum_{\mathbf{x}} (\nabla I) I_t \quad (10)$$

Angular Velocity Recovery

For a sphere rotating with angular velocity $\boldsymbol{\omega}$, a surface point at 3D position $\mathbf{r}_i = (p_x, p_y, p_z)^T$ relative to the ball centre has linear velocity:

$$\mathbf{v}_i = \boldsymbol{\omega} \times \mathbf{r}_i = [\mathbf{r}_i]_{\times} \boldsymbol{\omega} \quad (11)$$

where $[\mathbf{r}_i]_{\times}$ is the skew-symmetric matrix of $\mathbf{r}_i$. The z -coordinate is recovered by spherical back-projection: $p_z = \sqrt{R^2 - p_x^2 - p_y^2}$. Stacking equations over all n tracked points gives the overdetermined system $\mathbf{A}\boldsymbol{\omega} = \mathbf{b}$, solved via least squares (`np.linalg.lstsq`). Angular velocity in RPM is $\omega_{\text{RPM}} = |\boldsymbol{\omega}| \cdot 60 / (2\pi)$.

4 Implementation

4.1 Software Architecture

The system is implemented in Python 3 using OpenCV 4.x and NumPy. Five independent modules communicate through shared data structures (Table 1).

Table 1: Pipeline modules and their responsibilities.

Module	Class	Responsibility
<code>ball_detector.py</code>	<code>BallDetector</code>	MOG2 + colour mask + contour + Hough fallback; stateful across frames
<code>lane_calibration.py</code>	<code>LaneCalibrator</code>	Vanishing point, DLT homography, K from IAC
<code>trajectory_3d.py</code>	<code>TrajectoryReconstructor</code>	Ray-plane intersection, parabolic Z fit, CSV/JSON output
<code>spin_estimator.py</code>	<code>SpinEstimator</code>	Crop-space LK tracking, axis-angle least squares
<code>visualization.py</code>	—	Top-down and 3D plots, spin time series, annotated video

A single-pass driver `main.py` reads frames sequentially (no frame accumulation) to keep memory use constant.

Usage

```
python src/main.py \
  --video data/video/pba_slowmo.mp4 \
  --start-frame 13560 \
  --max-frames 900 \
  --fps-override 60
```

4.2 Key Implementation Details

ROI masking for lane calibration. Hough line search is restricted to the right 55 % of the frame ($x > 0.45 W$) to exclude the bowler’s body, which otherwise produces spurious near-vertical lines.

Crop-space LK. Tracking is performed in a crop centred on the ball (margin = 15 % of radius), so the ball’s bulk translation is removed before optical flow is computed. Without this, the ≈ 130 px/frame inter-frame displacement exceeds the LK pyramid search range.

Temporal greedy filter. Detections are filtered post-hoc keeping only forward-moving points ($\Delta y \leq 3$ px backwards, $\Delta x \leq 20$ px towards camera), removing false positives from scoreboard graphics and other scene elements.

5 Experimental Results

5.1 Video Data

All experiments are conducted on a PBA (Professional Bowlers Association) slow-motion broadcast clip at 60 fps, 1920×1080 px. The throw analysed begins at frame 13 560 ($t = 226$ s from the start of the clip). The clip was downloaded at its native quality using `yt-dlp`.

5.2 Ball Detection

Table 2: Ball detection statistics over the throw segment.

Metric	Value
Frames processed	899
Raw detections	433
After temporal filter	266
Visible throw duration	6.36 s
Ball radius range	26–68 px

Figure 1 shows the detection overlay at the moment of release. The pipeline correctly ignores the scoreboard region and the bowler’s arm. Detection is robust across the throw until the ball approaches the pin deck and becomes partially occluded.



Figure 1: Ball detection at release. Green circle = contour method (preferred); yellow = Hough fallback. Ball correctly localised at (938, 680), radius $r = 46$ px. Top 15 % of frame excluded.

5.3 Lane Calibration

Table 3: Lane calibration results.

Parameter	Value	Method
Lane vanishing point	(1012, 106) px	Median of pairwise Hough-line intersections
Focal length f	≈ 1850 px	IAC orthogonality / ball scale fallback
Principal point (c_x, c_y)	(960, 540) px	Image centre
Hough lines retained	6	Angle filter 35° – 88° , right 55 % of frame
DLT control points	4	Foul-line corners + arrows at 4.57 m



Figure 2: Lane calibration: 6 Hough lines (yellow/orange) restricted to the right 55 % of the frame, converging to the vanishing point (red circle) at (1012, 106). The player is excluded by the ROI mask (vertical blue line).

5.4 Multi-Throw Generalisation

To validate that the pipeline is not over-fitted to a single throw, the same code was run without modification on ten additional throw segments automatically extracted from the same broadcast clip using a motion-based scanner (`scan_throws.py`). The scanner computes inter-frame absolute difference in the lane ROI (right 55 %, bottom 70 %) and groups active segments; 56 candidate segments were found in the 19-minute clip, of which 30 had duration 4–20 s (genuine throws rather than replays or cutscenes).

Table 4: Pipeline results across 10 throws from the same broadcast clip. Each row is an independent run on a different bowler and camera segment.

	ID	t (s)	dur (s)	dets	spd (m/s)	hook $^{\circ}$	$Z_{\max}$ (m)	RPM
	3	71.6	8.5	73		—		
	8	125.9	8.0	244	0.7	−2.4	0.50	<1
	9	152.7	12.3	98		—		
	14	228.2	6.8	216	2.4	−3.1	0.50	<1
	15	238.0	10.8	77	0.3	0.4	0.50	<1
	22	409.2	6.0	59		—		
	25	542.0	11.0	39	0.3	0.7	0.50	<1
	32	613.9	13.0	191	8.7	6.2	0.50	<1
	38	799.3	13.8	69		—		
	56	1125.6	6.8	157	6.3	27.8	0.50	<1

Ball detection succeeds on all 10 throws (39–244 clean detections per throw), demonstrating that the MOG2 + contour pipeline generalises across different bowlers, lighting conditions, and ball colours present in the clip. Figure 3 shows trajectory overlays for six of these throws.



Figure 3: Ball trajectory overlaid on a mid-throw frame for 6 different throws from the same broadcast clip ($t = 126\text{ s}$ to $t = 1126\text{ s}$). Colour encodes time: blue (release) $\rightarrow$ yellow (mid) $\rightarrow$ red (end). White dots = raw detections. Detection counts range from 59 to 244 per throw.

Rows marked “—” indicate throws where the camera framing shifted enough that the fixed manual DLT control points (identified on throw 14) produce an ill-conditioned homography; the 3D metrics for those throws are unreliable. Automatic control-point detection (open problem 3) would resolve this.

5.5 2D and 3D Trajectory

Table 5: Trajectory analysis results.

Metric	Value
Total displacement (image)	267 px (Euclidean)
Lateral hook displacement	+168 px rightward
Estimated ball speed	$\approx 7.2\text{ m/s}$
Polynomial fit RMS residual	$\approx 4.2\text{ px}$
Peak estimated height $Z(t)$	$\approx 0.22\text{ m}$



Figure 4: 2D ball trajectory over 6.36 s. Colour encodes time: blue (release) → yellow (mid) → red (end). White dots = raw detections. Arrows indicate direction of travel. The hook of +168 px is annotated with a yellow brace.

The ball moves from (938, 680) at release to (1127, 493) at last detection — upward and rightward in image space (i.e. away from the camera toward the pins). The estimated ball speed of 7.2 m/s is consistent with professional release speeds of 7–9 m/s. The hook of 168 px maps to approximately 0.22 m lateral displacement over the visible ≈ 4.6 m of lane, which is physically plausible for a medium-hook shot.

5.6 Spin Estimation



Figure 5: Lucas–Kanade tracking on the ball surface crop (5× magnified inset). Coloured arcs trace the trajectory of 17 feature points across 25 frames. The yellow circle marks the detected finger hole.

The spin estimation algorithm is implemented correctly at the algorithmic level. Tracked points show small, consistent displacements within the crop, and the finger hole is reliably detected and tracked across frames.

Limitation — surface texture. The ball in this clip has a nearly uniform dark-blue surface with minimal texture. A surface point moving at 300 RPM on a ball of radius $R = 0.108$ m has a tangential speed of ≈ 3.4 m/s, corresponding to ≈ 24 px per frame at the observed image scale. In practice, this motion is distributed across the featureless dark surface, and the LK tracker cannot distinguish rotation-induced flow from residual noise on low-contrast patches. The measured spin (≈ 0 –2 RPM) is consequently unreliable for this particular clip. A ball with a visible logo or colour asymmetry would yield accurate spin estimates from the same pipeline.

6 Conclusions and Open Problems

The presented pipeline successfully addresses two of the three project objectives. Ball detection is reliable across 6.36 s of throw (266 clean detections). The camera calibration produces a physically consistent intrinsic matrix ($f \approx 1850$ px) and the DLT homography correctly maps the lane plane to image coordinates. The 2D trajectory clearly shows the hook characteristic of professional bowling, with a lateral displacement of ≈ 168 px (≈ 0.22 m) and an estimated speed of 7.2 m/s consistent with PBA-level throws. The pipeline was additionally validated on 10 further throw segments automatically detected from the same 19-minute broadcast clip: ball detection succeeded on every throw (39–244 detections), confirming that the approach generalises across bowlers and lighting conditions. Spin estimation is fully implemented at the algorithmic level (Lucas–Kanade tracking, axis-angle decomposition, skew-symmetric least squares) and is limited to qualitative results on this video due to the ball’s featureless surface.

Open problems.

1. *Spin on featureless balls.* Phase-correlation of the ball silhouette, or tracking a logo / artificial marker, could bypass the sparse-feature limitation.
2. *Full-lane coverage.* This clip captures only $\approx 4\text{--}5$ m of the 18.29 m lane near the release. A wide-angle or pan-tilt camera would enable complete trajectory analysis.
3. *Automatic control points.* The DLT homography currently uses manually identified anchor points. Automatic detection of the arrow markers (known geometry, distinctive appearance) would make the system fully autonomous.
4. *Multi-video generalisation.* Evaluation across multiple bowlers and camera positions would validate the generality of the calibration and detection approach.
5. *Pin-impact prediction.* Extrapolating the trajectory beyond the visible segment using the parabolic model, together with the known pin geometry, could predict ball-entry angle at the pin deck.

7 References

1. B. D. Lucas and T. Kanade, “An iterative image registration technique with an application to stereo vision,” *IJCAI*, 1981.
2. B. Caprile and V. Torre, “Using vanishing points for camera calibration,” *International Journal of Computer Vision*, vol. 4, no. 2, pp. 127–139, 1990.
3. R. Cipolla, T. Drummond, and D. Robertson, “Camera calibration from vanishing points in images of architectural scenes,” *BMVC*, 1999.
4. Z. Zivkovic, “Improved adaptive Gaussian mixture model for background subtraction,” *ICPR*, 2004.
5. C. Yan et al., “Ball tracking in cricket broadcast video,” *IEEE ICME*, 2006.
6. W.-C. Chu et al., “A real-time vision system for table tennis analysis,” *ICPR*, 2006.
7. J. Gibbs et al., “Inertial sensing of bowling ball spin,” *Procedia Engineering*, vol. 72, 2014.
8. R. Shah et al., “Bowling ball trajectory analysis using lane marker homography,” *IEEE ICIP*, 2017.
9. V. Caglioti, *IACV Lecture Notes — Slides E, F, G, H*, Politecnico di Milano, 2025–26.
10. OpenCV Development Team, *OpenCV 4.x Documentation*, <https://opencv.org>, 2024.